Semantic and Instance Segmentation

Per-pixel labels: FCN, U-Net, Dice, and Mask R-CNN

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

From boxes to pixels

The computer-vision task ladder

Each task asks **more** of the same image.

last lecture ended at **detection**; today we go all the way to **every pixel**.

Recap: what detection gave us

For each object, detection returns a **box + class + score**:

$$x \quad \rightarrow \quad \left\{ \left(\hat{b}_i, \hat{y}_i, \hat{s}_i \right) \right\}_{i=1}^{N}$$

- $\hat{b}_i$ — a **rectangle** (x, y, w, h);
- $\hat{y}_i$ — the class;
- $\hat{s}_i$ — a confidence, filtered by **NMS**.

A box says **"an object is roughly here."** It never traces the object's actual **shape**.

The limit of a box

- a box is **axis-aligned** — a diagonal or curved object wastes most of it;
- overlapping objects share pixels — which pixel belongs to whom?
- some questions need the **outline**: tumor area, road surface, free space.

we want a label for every pixel, not a rectangle

Segmentation: label every pixel

V

a toy 6 × 8 label map — one class per pixel

the output has the **same spatial size** as the input — one class per pixel.

Semantic segmentation — the task

Assign a class to **every** pixel (h, w):

$$x \quad \rightarrow \quad y_{hw} \in \{1, ..., K\} \quad \forall\, h, w$$

- output is a **label map** of size $H \times W$;
- **all** pixels of the same class look identical — two cats merge into one "cat" region;

Semantic segmentation answers **"what class is this pixel?"** — it does **not** count objects.

Instance segmentation — the task

Return a **mask per object**, like detection but with pixels:

$$x \quad \rightarrow \quad \left\{ \left(\hat{y}_i, \hat{b}_i, \hat{m}_i, \hat{s}_i \right) \right\}_{i=1}^{N}$$

- $\hat{m}_i$ — a **binary mask** over the object's pixels;
- two cats now become **two separate** masks — objects are **counted and separated**.

detection's boxes, upgraded to pixel-accurate masks

Panoptic segmentation — one label for everything

Give **every** pixel a pair — a class **and** an instance id:

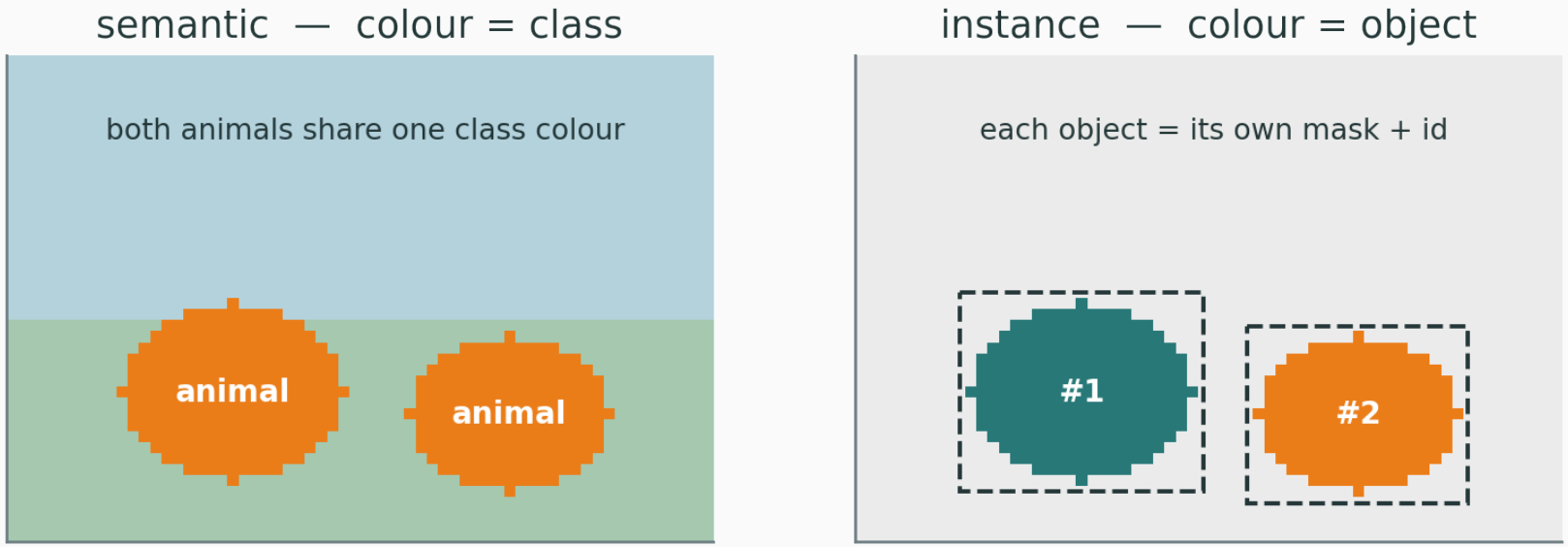
$$x \quad \rightarrow \quad (c_{hw}, \mathrm{id}_{hw}) \quad \forall\, h, w$$

stuff — amorphous regions (sky, road, grass): class only, **no instance**.

things — countable objects (car, person): class **and** a distinct instance id.

panoptic = semantic (stuff) **+** instance (things), with **no gaps or overlaps**.

Semantic vs instance, side by side



same scene · **left**: colour by class · **right**: colour by object.

The spine of this lecture

Detection gives **boxes**;
segmentation labels every pixel.

Task	**Output**	**Counts objects?**
semantic seg	per-pixel class map	no
instance seg	per-object mask	yes
panoptic seg	per-pixel (class, id)	yes (things only)

Semantic segmentation

Per-pixel classification

Segmentation is **classification, run at every pixel**:

$$p_{hwk} = p(y_{hw} = k \mid x)$$

- for each pixel (h, w) the network emits a vector of K class scores;
- a **softmax over the K channels** turns scores into probabilities;

Same loss you already know from image classification — just applied $H \times W$ times, once per pixel.

The output is a tensor

A classifier outputs **one** vector of length K. Segmentation outputs **a grid** of them:

$$\text{logits} \in \mathbb{R}^{H \times W \times K} \quad \rightarrow \quad \text{softmax over} \quad K \quad \rightarrow \quad p \in \mathbb{R}^{H \times W \times K}$$

the **channel** dimension holds class probabilities; argmax_k per pixel gives the label map.

Pixelwise cross-entropy

D

Sum the ordinary cross-entropy over **all pixels** and average:

$$\mathcal{L} = -\frac{1}{HW} \sum_{h,w} \log p_{hw,\, y_{hw}}$$

- y_{hw} — the **true** class at pixel (h, w);
- $p_{hw,\, y_{hw}}$ — the probability the model gave to that **correct** class;

each pixel contributes its own classification loss

Reading the loss

$$\mathcal{L} = -\frac{1}{HW} \sum_{h,w} \underbrace{-\log p_{hw,\, y_{hw}}}_{\text{loss at one pixel}}$$

- a **confident, correct** pixel ($p \to 1$) contributes ≈ 0;
- a **confident, wrong** pixel ($p \to 0$) contributes a **large** penalty;

the $\frac{1}{HW}$ makes it a **mean** — it treats every pixel as equally important (we will question this soon).

We need dense prediction

- a classification CNN ends in **pooling + fully-connected** layers → one vector;
- that **destroys spatial layout** — exactly what a pixel map needs to keep.

how do we turn a "one-label" network into an "$H \times W$-label" network?

Fully convolutional networks (FCN)

Replace the final **fully-connected** layers with 1×1 **convolutions**:

- a 1×1 conv is an FC layer applied **at every spatial location**;
- the net now accepts **any input size** and emits a **coarse label map**, not a vector;

"Fully convolutional" = **no FC layers at all**. Output is a spatial map of class scores — the first end-to-end dense-prediction net.

FCN: coarse semantics meet fine detail

Deep layers know **what**; shallow layers know **where**.

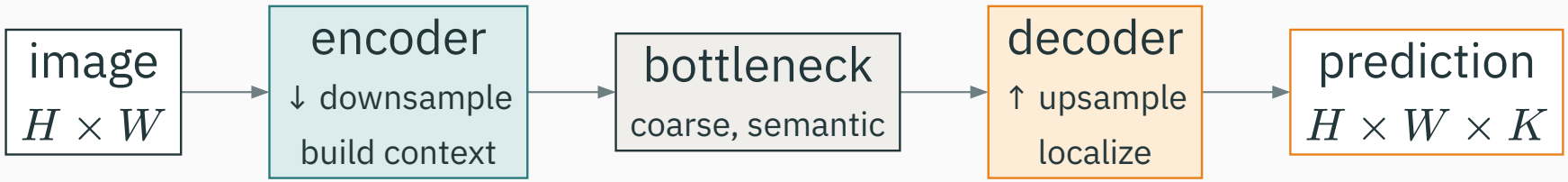
Deep features — rich **semantics**, but **coarse** (heavily downsampled).

Shallow features — precise **location / edges**, but weak semantics.

FCN **fuses** a coarse deep prediction with shallow appearance to sharpen boundaries.

Encoder–decoder

The general shape of every modern segmentation net:



encoder shrinks space to gather context · **decoder** grows it back to pixel resolution.

Upsampling in the decoder

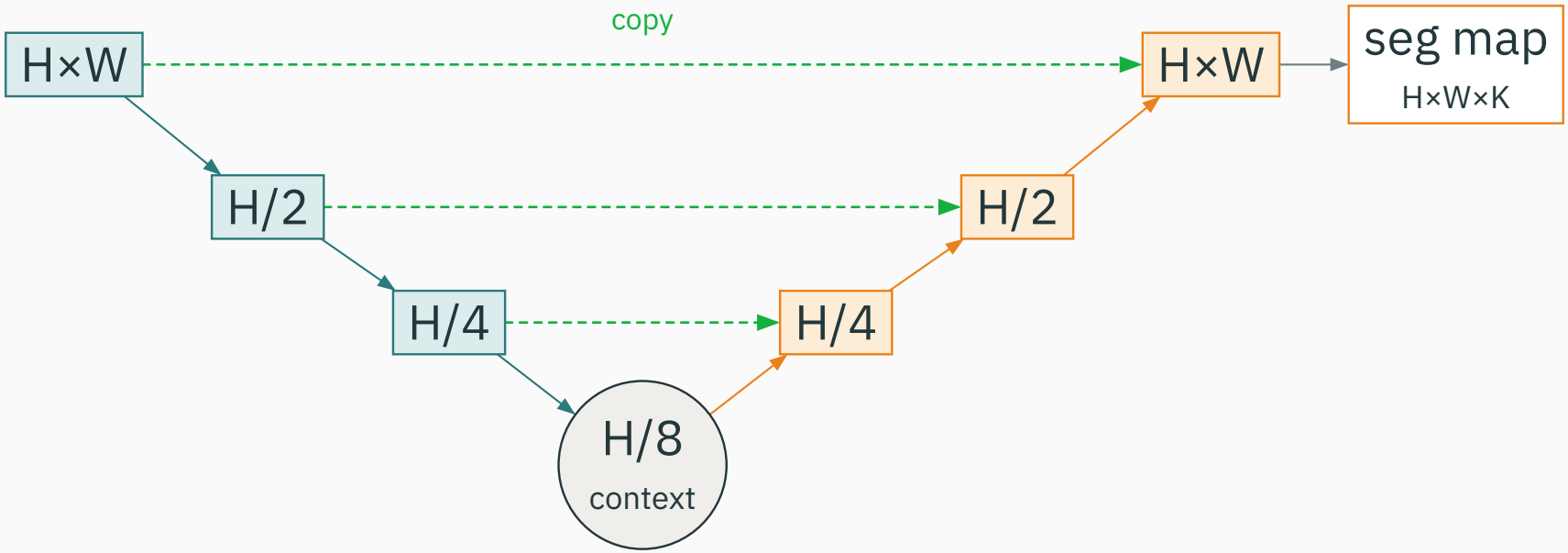
The decoder must **increase** spatial resolution. Two common ways:

- **transposed convolution** — a learned upsampling (“deconv”);
- **bilinear upsample + conv** — cheap interpolation, then refine.

Downsampling was **pooling / stride**; upsampling **undoes** it — but the fine detail that pooling threw away is gone unless we bring it back another way.

U-Net

Symmetric encoder–decoder with **skip connections** at every level:



contracting path (teal) captures context · expanding path (orange) localizes · green skips copy detail across.

Why skip connections

The bottleneck knows **what** but has lost **where**. Skips restore the **where**.

- each skip **copies** high-resolution encoder features to the matching decoder level;
- the decoder **concatenates** them with its upsampled features before predicting;

coarse semantics (from depth) + fine detail (from skips) = sharp masks

U-Net: born in medical imaging

optional

- designed for **biomedical** segmentation, where images are large and labels scarce;
- works from **very few** annotated images (heavy augmentation);
- the skip-connected encoder–decoder is now the **default backbone** for dense prediction — including diffusion models.

Interactive: encoder–decoder / U-Net

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/unet

Walk an image down the contracting path and back up the expanding path — toggle the skip connections and watch the predicted mask lose, then recover, its fine boundaries.

Q. If you **remove** every skip connection, what happens to the sharpness of the output mask, and why?

Losses for dense prediction

The class-imbalance problem

In dense prediction the **background usually dominates** the image.

- a small tumor may be **< 1%** of the pixels; the rest is “healthy”;
- pixelwise CE is a **mean over pixels** → the majority class drowns the minority.

A network can score **low CE and high pixel accuracy** by predicting **“all background”** — and still be useless.

The Dice coefficient

Measure **overlap** between predicted set P and ground-truth set G:

$$\text{Dice} = \frac{2|P \cap G|}{|P| + |G|}$$

- $= 1$ when P and G coincide exactly; $= 0$ when they are disjoint;
- it **ignores** the vast background — it only cares about the **foreground overlap**.

same idea as the F1 score, phrased for masks.

Soft Dice loss

Make Dice differentiable: use **probabilities** $p_i \in [0, 1]$ instead of hard sets, and **minimize** $1 - \text{Dice}$:

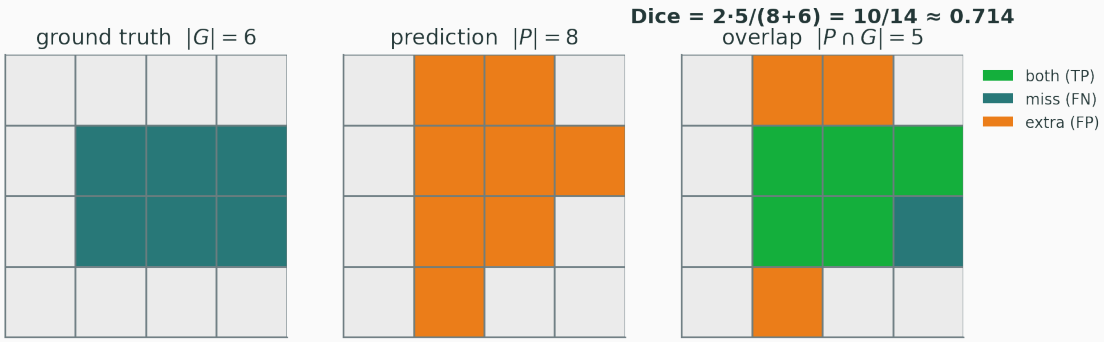
$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i p_i g_i + \varepsilon}{\sum_i p_i + \sum_i g_i + \varepsilon}$$

- p_i — predicted foreground probability at pixel i; $g_i \in \{0, 1\}$ — ground truth;
- ε — a small constant so an empty mask is **stable**, not $\frac{0}{0}$.

gradient pushes overlap up directly, regardless of how much background there is

Worked Dice example

D



On a 4×4 grid:

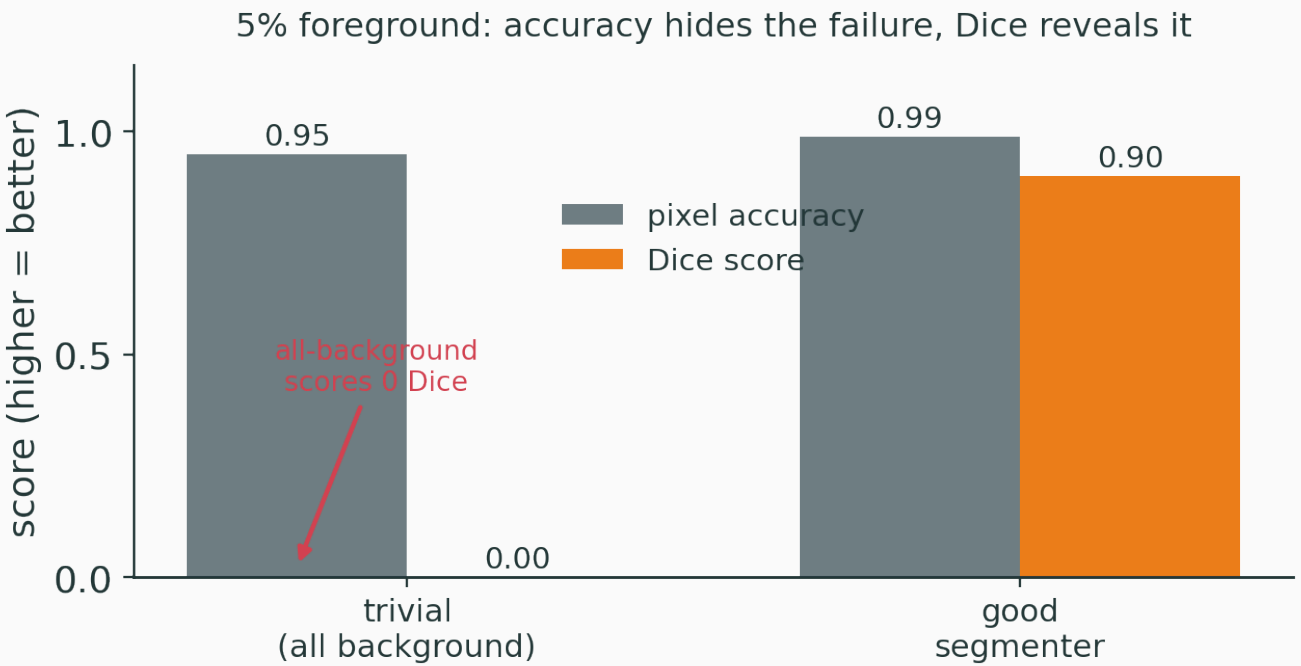
- $|G| = 6$ (true region),
- $|P| = 8$ (predicted),
- $|P \cap G| = 5$ (overlap).

$$\text{Dice} = \frac{2 \cdot 5}{8 + 6} = \frac{10}{14} \approx 0.714$$

green = correct (TP) · teal = missed (FN) · orange = extra (FP).

When Dice helps

Imbalanced scene (5% foreground), two “models”:



pixel accuracy calls both **“great”**; Dice **exposes** the trivial all-background predictor.

CE vs Dice — use both

They fail in **opposite** ways, so practitioners **combine** them:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \, \mathcal{L}_{\text{Dice}}$$

Cross-entropy — smooth, stable gradients per pixel; but **imbalance-blind**.

Dice — directly targets **overlap**; but noisier gradients, unstable on empty masks.

CE gives a clean training signal; Dice keeps the **foreground** honest.

Interactive: segmentation losses

I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/image-segmentation

Paint a prediction over a ground-truth mask and watch pixel CE and the Dice score move — see how a mostly-background prediction can keep CE low while Dice collapses to zero.

Q. Why does the trivial "predict all background" solution give **Dice** $= 0$ even though its pixel **accuracy** is 95%?

Instance segmentation

From semantic to instance

Semantic seg cannot **separate** two touching cats — both are just “cat” pixels.

instance seg = detect each object, then segment inside its box

so we bolt a **mask** onto a detector — one mask per detected object.

Recap: Faster R-CNN

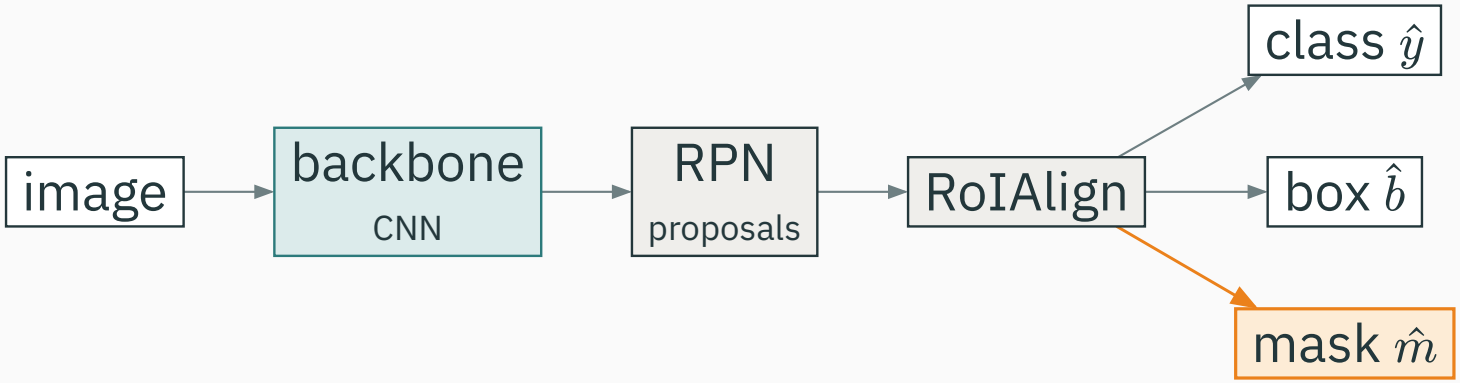
The two-stage detector we build on:

$$\text{backbone} \rightarrow \text{RPN (proposals)} \rightarrow \text{RoI features} \rightarrow \{\text{class}, \text{box}\}$$

- the **RPN** proposes candidate regions;
- each region's features feed **two** heads: a **class** head and a **box-regression** head.

Faster R-CNN already localizes objects — it just stops at boxes.

Add a **third, parallel** head that predicts a **mask**:



same backbone + RPN · three heads: **class**, **box**, and the new **mask** branch.

The mask branch

For each RoI, a **small FCN** predicts one binary mask per class:

- output is K masks of size $m \times m$ (e.g. 28×28), one per class;
- the **classification head chooses which** mask to use — the mask branch stays **class-agnostic** in its loss;

Decoupling **"what class"** from **"which pixels"** is the key trick: masks and classes do not compete.

Mask loss

D

Per-pixel **binary** cross-entropy, only on the RoI's chosen-class mask:

$$\mathcal{L}_{\text{mask}} = -\frac{1}{m^2}\sum_{u,v}[\, m_{uv}\log\hat{m}_{uv} + (1-m_{uv})\log(1-\hat{m}_{uv})\,]$$

- $m_{uv} \in \{0,1\}$ — true mask; $\hat{m}_{uv} \in [0,1]$ — predicted (per-pixel **sigmoid**);
- summed only over the $m \times m$ RoI, and only for the **ground-truth class**.

total loss: $\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}}$

RoIAlign — why alignment matters

The old **RoIPool rounded** region coordinates to the feature grid.

- rounding shifts features by up to **half a cell** — invisible for a coarse box;
- for a **per-pixel mask** that misalignment **smears the boundary**.

RoIAlign skips rounding: it samples features at **exact** sub-pixel locations with bilinear interpolation — the single change that made masks sharp.

Summary

Task → loss, at a glance

Task	Output	Loss
semantic seg	per-pixel class map	pixel CE (+ Dice)
instance seg	per-object mask	detection loss + mask BCE
panoptic seg	per-pixel (class, id)	both, fused

the **output shape** dictates the **loss**.

Segmentation methods

Method	**Core idea**	**Gives**
FCN	FC $\rightarrow 1 \times 1$ conv: dense conversion	coarse class map
U-Net	encoder–decoder + skip connections	sharp class map
Mask R-CNN	detector + parallel mask branch	per-object masks

FCN made dense prediction possible; U-Net made it **sharp**; Mask R-CNN made it **per-object**.

Where segmentation is used

optional

- **medical imaging** — tumor / organ delineation (U-Net's home turf);
- **autonomous driving** — drivable surface, lanes, pedestrians (panoptic);
- **photo editing** — background removal, object selection;
- **remote sensing** — land cover, buildings, roads from satellite images.

Final mental model — one idea

Classification asks **what?**

Detection asks **what + where (box)?**

Segmentation asks: what at each pixel?

coarser → finer: one label · a few boxes · a whole grid of labels

Detection gives boxes; segmentation labels every pixel.

From grids of pixels to sequences of tokens — next, **sequence models**.